CSE_3523

Segment-7 (Part-2)
Embedded system

Microprocessor, microcontroller and embedded system

Course code: CSE-3523
Credit Hour: 3

Md. Mujibur Rahman Maruf

Lecturer,

Dept. of CSE,IIUC

Mujiburmaruf.cuet17@gmail.com

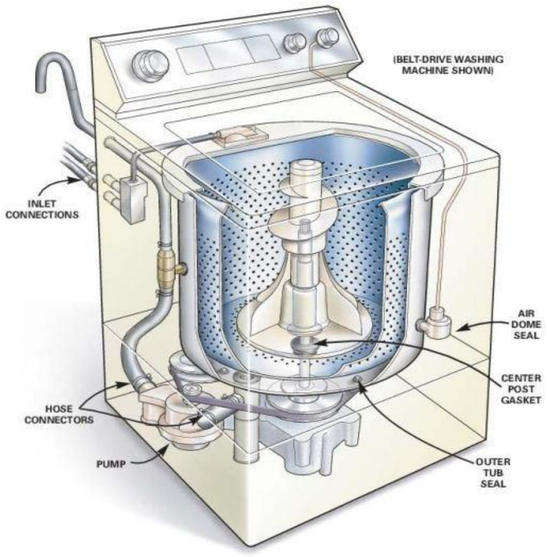
International Islamic University Chittagong(IIUC)

SYSTEM

What is a System?

•A **system** is something that **organizes or performs tasks** according to a **fixed plan, program, or set of rules**.

•It includes **multiple parts** that work **together in coordination** to complete a function.



Example of a System: Washing Machine

A washing machine is an **automatic system** that cleans clothes by performing tasks in a **predefined sequence** using various components

Embedded System

- An **Embedded System** is a **computer system designed to perform a specific, dedicated function**, often as part of a larger system. It is built with a processor, memory, input/output units, and software (firmware)
- **Embedded** refers to a system that is **placed inside** another device or system to perform a **specific task or function**.

Example:

In a **washing machine**, there is a small computer (embedded system) **inside** that controls the washing process.

That computer is **embedded** within the machine — it **is not visible** but **essential** for operation.

Types of Embedded Systems

Embedded systems are **categorized based on complexity and functionality**. Understanding these classifications helps in choosing the right development tools and architectures:

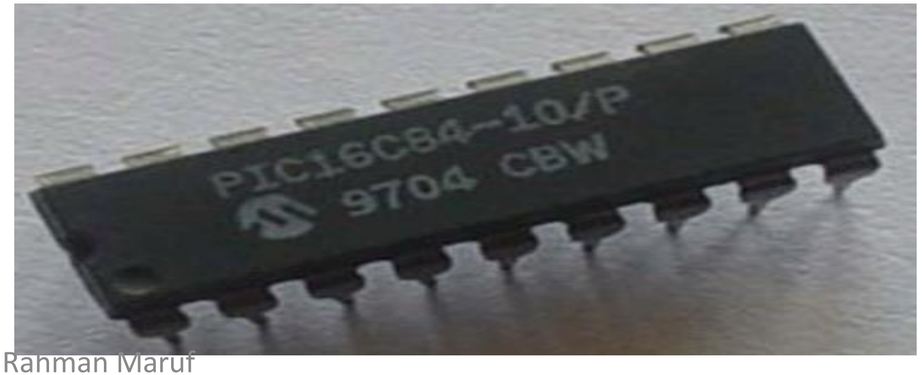
1.Small-Scale Embedded Systems:

Designed using 8-bit or 16-bit microcontrollers, these systems are **simple**, **cost-effective**, and often **battery-powered**. Software is written in C or assembly and must fit within limited memory and power constraints

Example:

Digital watches

Simple calculators



.

Types of Embedded Systems

2.Medium-Scale Embedded Systems:

These systems use **16 or 32-bit microcontrollers**, **DSPs**, or **RISC processors** and involve moderate hardware and software complexity. They often use **RTOS**, for development.

Example:

Home security systems Industrial automation controllers



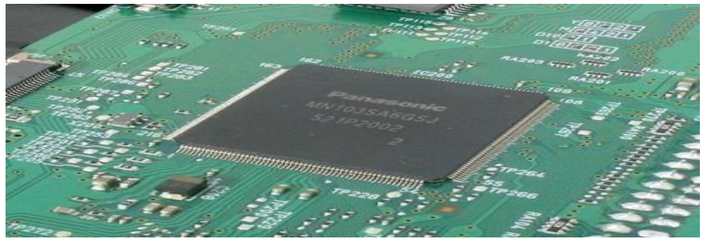
3.Sophisticated Embedded Systems:

Used in **high-performance, real-time applications**, these systems have **large memory**, **powerful processors**, and require **hardware-software co-design**

Example: Self-driving car systems

Medical imaging devices (MRI, CT)

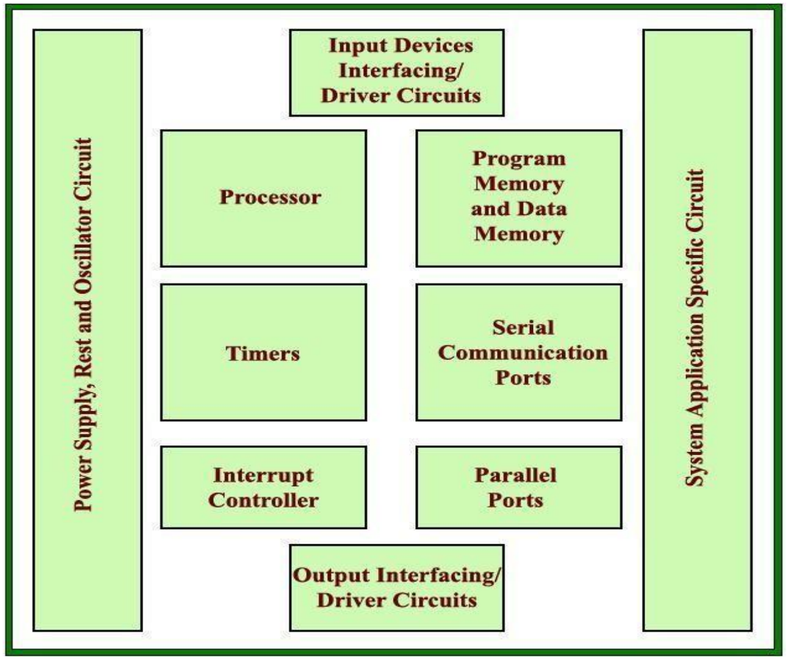
Satellite control systems



Elements of Embedded system

Hardware

The physical part of the embedded system that includes the processor, memory, input/output interfaces, sensors, and other electronic components.



Example: In a **digital thermometer**, the hardware consists of a temperature sensor (to sense temperature), a microcontroller (to process data), display (to show temperature), and power source (battery).

Elements of Embedded system

2.Software

The set of instructions (firmware) that runs on the hardware to perform specific tasks. It includes the embedded application code and device drivers.

Embedded application: The program controlling the embedded system's operation.

Device drivers: Interface between the software and hardware peripherals.

Example:

In the same **digital thermometer**, software reads the sensor data, converts it into temperature values, controls the display output, and manages user input (like switching between °C and °F).

Elements of Embedded system

3.RTOS (Real-Time Operating System)

A specialized operating system designed to run embedded applications that require real-time task scheduling and predictable response times.

Task Scheduling: Manages multiple tasks simultaneously.

Resource management: Efficient use of CPU, memory, and peripherals.

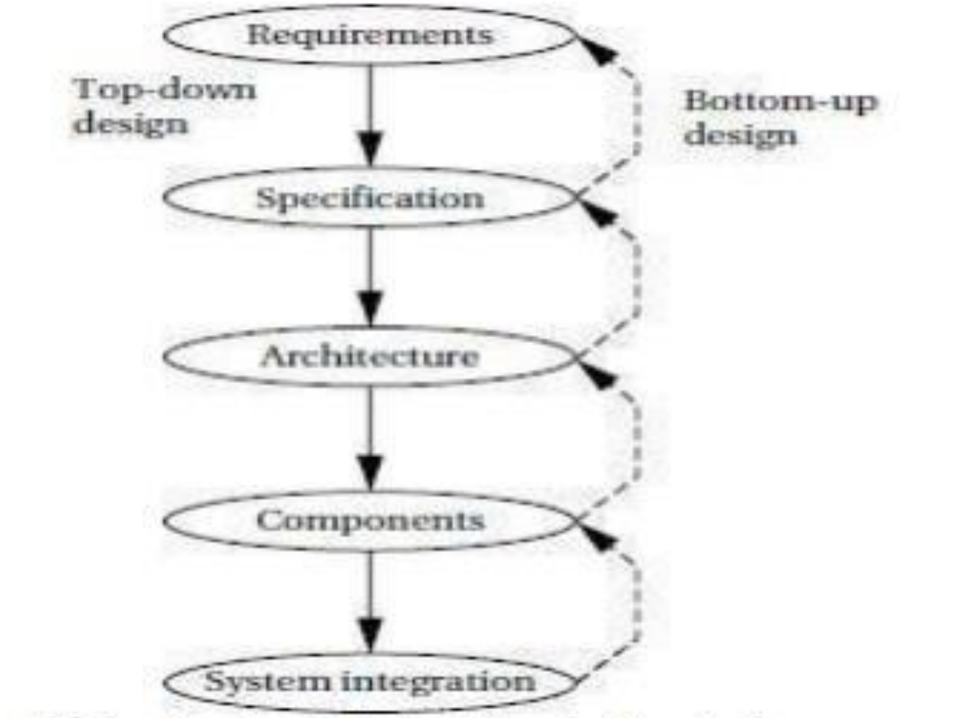
Interrupt handling: Responds quickly to events.

Example:

In a **car's airbag system**, an RTOS ensures the sensor data is processed immediately, and the airbag deploys without delay. It manages multiple real-time tasks like sensor monitoring, system diagnostics, and communication.

steps of embedded system design

- **Requirement**.
- **Specification**
- **Architecture**
- **Component Selection**.
- **Integration & Testing:**



: Major levels of abstraction in the design process

steps of embedded system design

1.Requirements: Define what the system must do

Example: For a smart thermostat, the system should control home temperature automatically, connect to Wi-Fi, display temperature, and consume low power.

2.Specification: Detail inputs, outputs, and performance criteria.

For the smart thermostat:

- Input: Temperature sensor readings.
- Output: Control signal to HVAC system, LCD display updates.
- Performance: Temperature reading update every 1 second.
- Constraints: Operate between 0°C to 50°C, battery life of 6 months.

steps of embedded system design

3.Architecture Design: Plan the hardware and software structure..

Example:

Hardware: Microcontroller, temperature sensor, Wi-Fi module, LCD display,

Software: Sensor reading module, control algorithm, user interface, communication protocol.

4.Component Selection:

Choose specific hardware and software components based on the architecture.

Example:

Microcontroller: ARM Cortex-M4 with built-in ADC.

Temperature sensor: Digital temperature sensor (e.g., DS18B20).

Wi-Fi module: ESP8266. Display: 16x2 LCD.

Software: FreeRTOS for task management, custom firmware.

steps of embedded system design

5. Integration and Testing

Combine components and verify system functionality

Example:

Assemble the PCB with selected components.

Load firmware and test temperature sensing, Wi-Fi connectivity,

display updates, and HVAC control.

Debug and optimize for performance and power consumption.

Applications of Embedded Systems

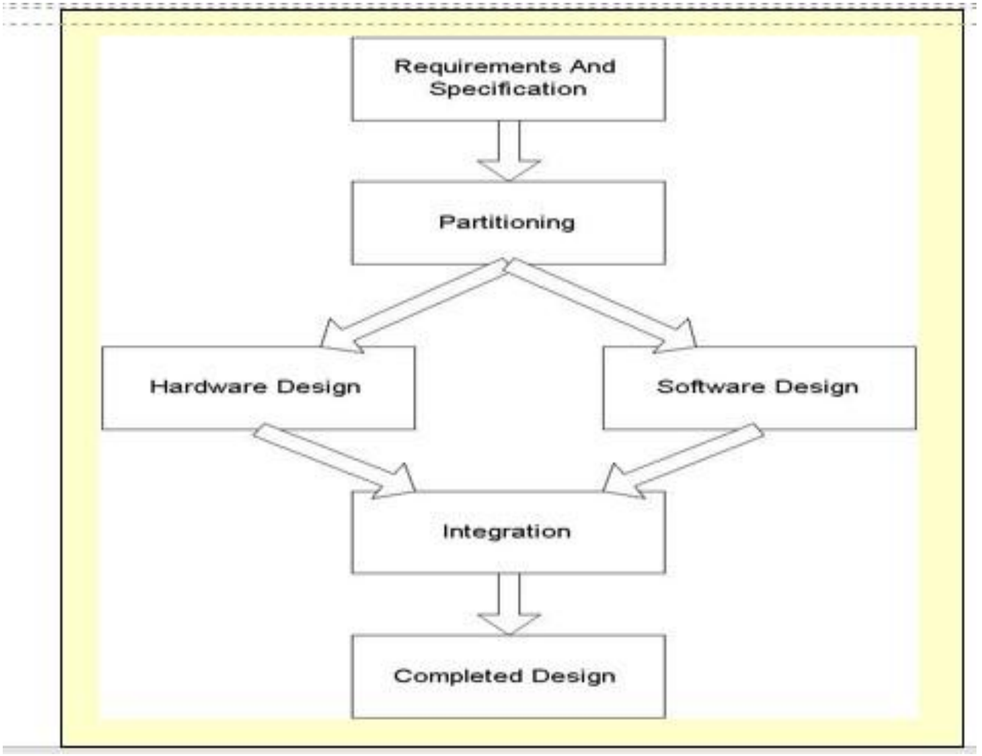
Embedded systems are at the heart of **modern technology**, embedded into a wide range of products. Their applications span across:

- **Telecommunication:** Routers, modems, mobile base stations.
- **Automotive:** Engine control, ABS, GPS systems.
- **Consumer Electronics:** TVs, MP3 players, digital cameras.
- **Industrial Automation:** PLCs, robotic arms, sensors.
- **Medical Devices:** Heart rate monitors, infusion pumps.

These systems are designed to **interact with the physical world**, sense inputs, and control outputs.

Traditional Embedded System Design

In traditional embedded system design, the process follows a **linear and rigid flow**. It begins with **requirements and specifications**, followed by an early **hardware/software (HW/SW) partitioning**. Once partitioning is done, the design proceeds in **two separate tracks** — hardware design and software design — usually handled by different teams. These components are then integrated in the final stage.



Key Characteristics:

- **Early HW/SW Partitioning:**

Partitioning is done at the beginning based on fixed assumptions. If any mistake is made in deciding what part goes to hardware or software, it becomes very hard to correct later in the process.

Traditional Embedded System Design

- **Independent Development:**

Hardware and software teams often work separately, with little interaction or feedback. This lack of synchronization can result in **compatibility issues** during integration.

- **Delayed Integration:**

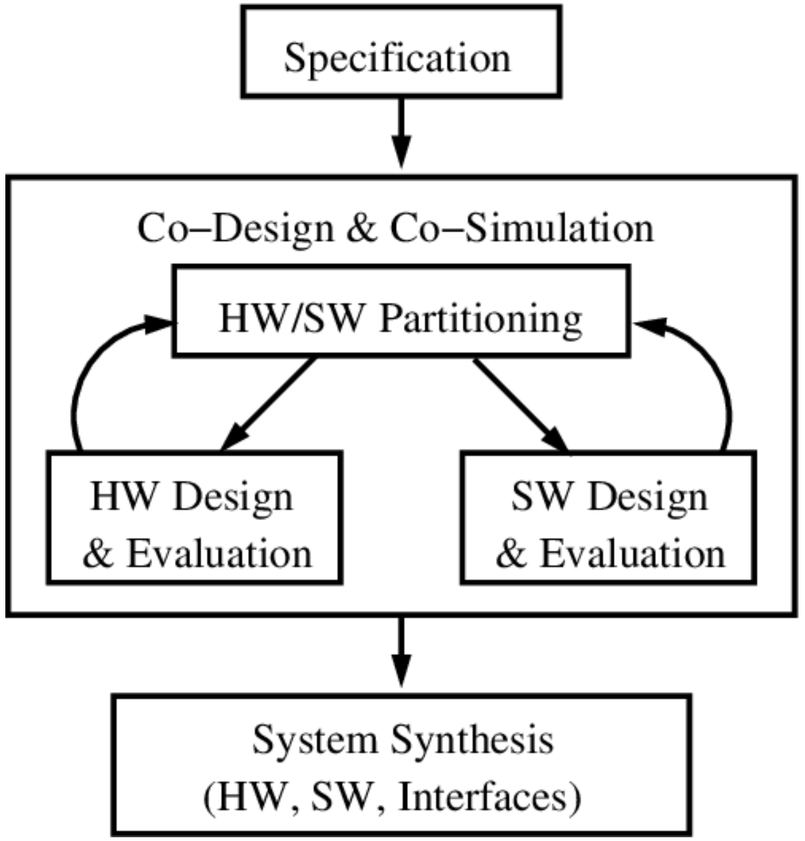
Integration is performed at the end, so **problems are discovered late**, and correcting them is costly and time-consuming.

- **Limited Flexibility:**

Once hardware is designed, it cannot be easily changed to fix a functional or performance issue without major redesign.

Hardware-Software Co-Design

Hardware-Software Co-Design takes a **collaborative and iterative** approach. The process begins with **system-level specification and analysis**, followed by **joint HW/SW architecture planning and partitioning**. Rather than being fixed early, the partitioning evolves throughout development based on functional, performance, and power requirements.



Hardware-Software Co-Design methodology

Hardware-Software Co-Design

Co-design of (embedded) computer systems encompassing the following parts

.

1. **Co-Specification**
2. **HW-SW Partitioning**
3. **HW-SW Co-Synthesis**
4. **HW-SW Co-Simulation**
5. **Co-Verification**

Hardware-Software Co-Design

1.Co specification:

Define the system's intended functionality at a high level using functional models (e.g., task graph).Focuses on **what** the system does, not how it's built.

Example: For home security system

Define key functions: detect motion/door opening, trigger alarm, send alert.

Model tasks like sensing, processing, and actuation in a task graph.

2.HW-SW Partitioning

Analyze the task graph and decide **which tasks to implement in hardware** and which in **software**.

Decisions based on **performance, cost, power, and flexibility**.

Example: Assign sensors (PIR, door contacts) and alarm hardware to hardware.

Assign intrusion logic, alert generation, and cloud communication to software..

Hardware-Software Co-Design

3. Hardware-Software Co-Synthesis

4 principal phase of co synthesis

A.Partitioning: Break the system into smaller parts or tasks.

Example: In a smart home system, Motion sensing → Intrusion logic → Activate alarm → Send notification.

B.Scheduling : Decide the order and timing for each task.

*Example:*Sense motion every second → Process immediately → Trigger alarm & send alert within 1 second.

Hardware-Software Co-Design

Hardware-Software Co-Synthesis

C.Allocation

Assign tasks to specific processing units (hardware or software).

Example: Motion sensing & alarm on hardware; decision logic & alerting on software

D.Mapping

Choose the exact hardware or software components for each assigned task.

- PIR sensor for motion (HW)
- ESP32 MCU running C code for processing (SW)
- Buzzer via GPIO for alarm (HW)
- Wi-Fi module for mobile alert (SW)

Hardware-Software Co-Design

4. HW-SW Co-Simulation

Simulate the combined HW-SW system **before physical prototyping**.

•Validates timing, control logic, and communication between modules.

•**Example:** Simulate sensor activation, alarm response, and alert flow.

Ensure real-time response and correct logic execution under different scenarios.

5. Co-Verification

Ensure that the **final integrated system meets specifications** using testing or formal verification methods.

Verifies functional correctness, timing, and interface reliability.

Example: Test for intruder detection, false alarm handling, and message delivery.

Verify all features work as specified before physical deployment.

Computational Model

What is a Computational Model?

•A computational model is a method to describe how a system behaves and processes information.

•Used to plan and test a system before building it in hardware or software.

•Acts like a **blueprint** for the system's operations

•In embedded system co-design, the computational model helps define whether a functionality should be implemented in **hardware**, **software**, or **both**, based on performance, cost, and other trade-offs.

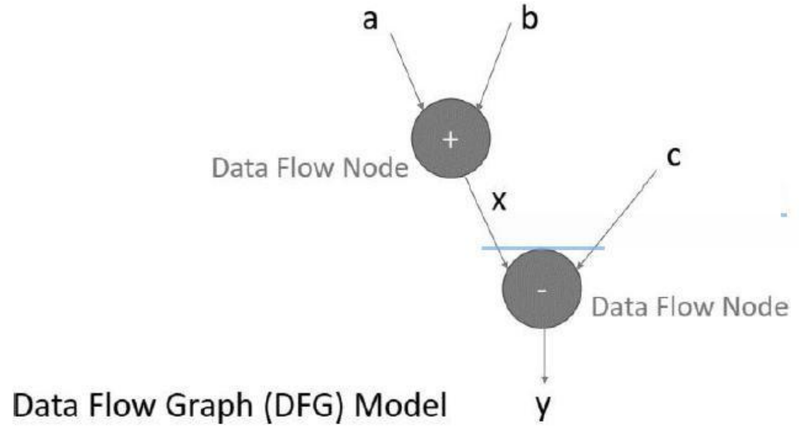
Computational Model

Data Flow Graph (DFG) Model

- **Data-driven execution**: actions occur when data is ready.
- Used in systems where **data processing is key**, like DSP.
- **Example**:

If a function computes x = a + b and y = x - c

- **Graph**:
  - Circles: represent operations (+, -)
  - Arrows: show flow from inputs a, b, c to outputs x, y
- **Use case**: Audio processing, image filtering.



Data Flow Graph (DFG) Model

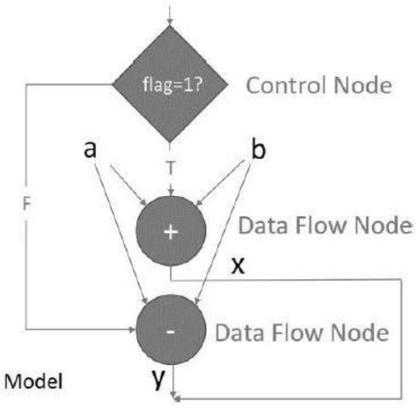
Computational Model

Control Data Flow Graph (CDFG) Model

- Extends DFG by adding **conditional logic** (e.g., if, else).
- Includes **data operations and decision blocks**.
- **Example**:

If flag = 1, then x = a + b; else y = a - b.

- **Graph**:
  - Decision (diamond block) splits flow
  - Different data paths based on condition
- **Use case**: Camera choosing JPEG or PNG format based on settings.



Control Data Flow Graph (CDFG) Model

Computational Model

Finit State Machine Model (FSM)

A **Finite State Machine (FSM)** is a **computational model** used to describe the behavior of systems that go through a sequence of **states**, based on **inputs or events**.

It consists of:

- **State**: A situation or step the system is in (example: "Waiting", "Running").
- **Event/Input**: Something that happens to cause a change (example: "Insert coin").
- **Transition**: The change from one state to another.
- **Action**: What the system does when it changes state (example: "Start alarm").

FSM Model – Example 1

design of an embedded system for driver/passenger 'Seat Belt Warning' in an automotive using the FSM model for following requirements

1. When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.
2. The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

Solutions:

Here the states are

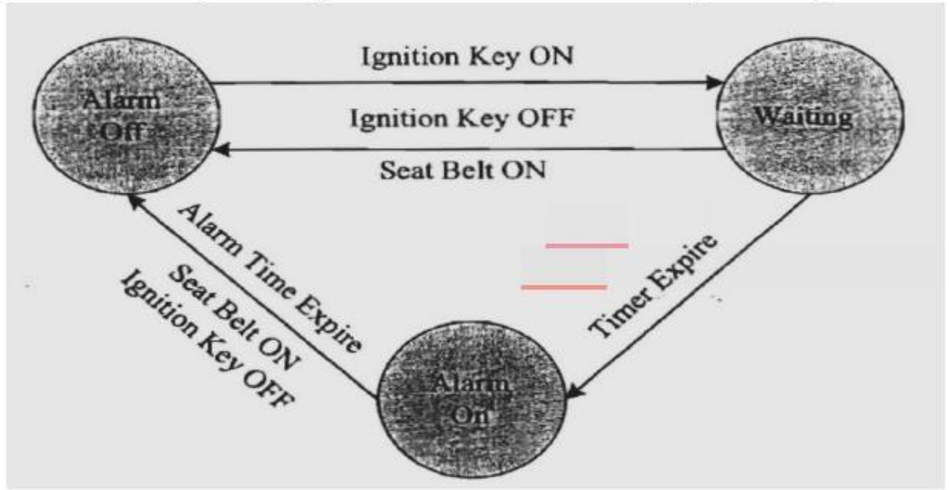
- 'Alarm Off’
- 'Waiting’
- 'Alarm On’

The events are

- 'Ignition Key ON’
- 'Ignition Key OFF’
- 'Timer Expire’
- 'Alarm Time Expire’
- 'Seat Belt ON’

FSM Model – Example 1

Using the FSM, the system requirements can be modeled as given in figure.



State Diagram

☐ The 'Ignition Key ON' event triggers the 10 second timer and transitions the state to 'Waiting'.

☐ If a Seat Belt ON' or 'Ignition Key OFF' event occurs during the wait state, the state transitions into 'Alarm Off'.

☐ When the wait timer expires in the waiting state, the event 'Timer Expire' is generated and it transitions the state to 'Alarm On' from the 'Waiting' state.

☐ The 'Alarm On' state continues until a 'Seat Belt ON' or 'Ignition Key OFF' event or 'Alarm Time Expire' event, whichever occurs first.

☐ The occurrence of any of these events transitions the state to 'Alarm Off'.

FSM Model – Example 2

Design an automatic tea/coffee vending machine based on FSM model for the following requirement.

- The tea/coffee vending is initiated by user inserting a 5 taka coin.
- After inserting the coin, the user can either select 'Coffee' or 'Tea' or press 'Cancel' to cancel the order and take back the coin.

Solution

The FSM Model contains four states namely,

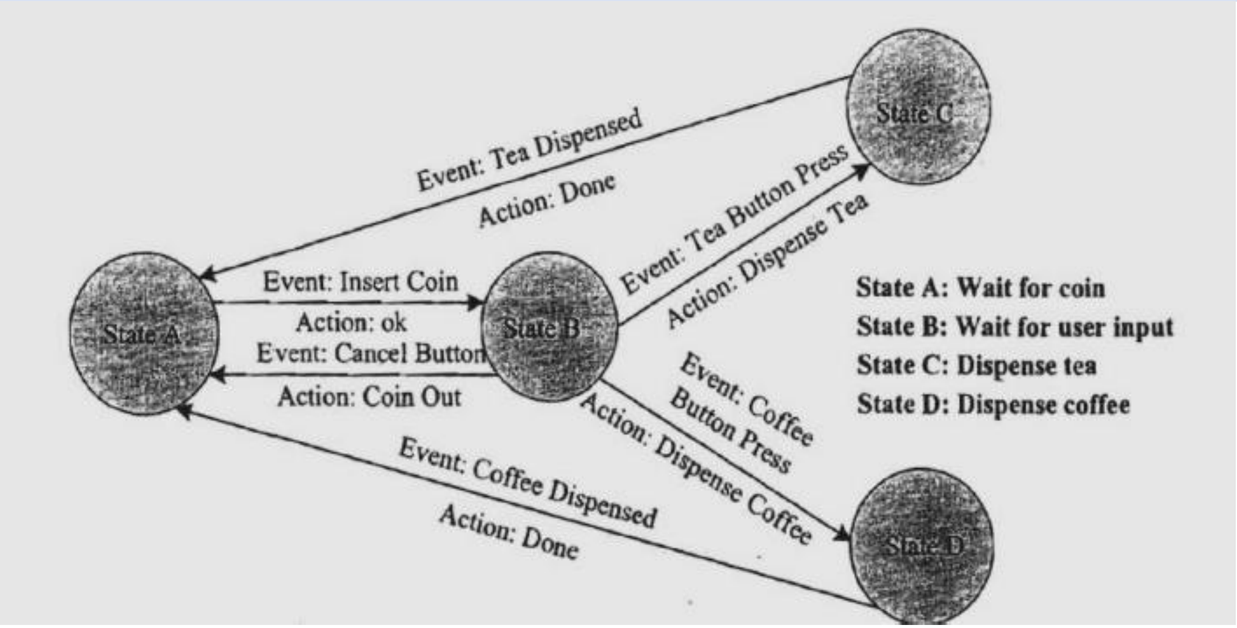
1. 'Wait for coin’
2. Wait for User Input’
3. Dispense Tea’
4. Dispense Coffee'

4 Events:

- Insert Coin
- Press Tea
- Press Coffee
- Press Cancel

FSM Model – Example 2

The event **'Insert Coin'** (5 taka coin insertion), transitions the state to **'Wait for User Input'.**



□ The system stays in this state until a user input is received from the buttons **'Cancel', 'Tea' or 'Coffee'**

□ If the event triggered in **'Wait State**' is **'Cancel'** button press, the coin is pushed out and the state transitions to **'Wait for Coin'.**

□ If the event received in the **'Wait State**' is either '**Tea**' button press, or '**Coffee**' button press, the state changes to '**Dispense Tea**' and '**Dispense Coffee**' respectively.

□ Once the coffee/tea vending is over, the respective states transition back to the **'Wait for Coin' state.**

FSM Model – Example 3

Design a Finite State Machine (FSM) model for a simple elevator (lift) controller in a multi-story building.

Requirements:

The elevator responds to floor requests and moves accordingly. When a request is received, the elevator moves either **up** or **down** to reach the requested floor. Once it reaches the requested floor, the **door opens** and stays open for at least **10 seconds**. The door remains open until a new request is received. The elevator must never have its door open while moving. The direction is only changed if there are no Pending requests in the current direction of motion

FSM Model – Example 3

Solution:

States:

The elevator can be in one of the following four states:

- Idle –
- GoingUp
- GoingDown
- DoorOpen

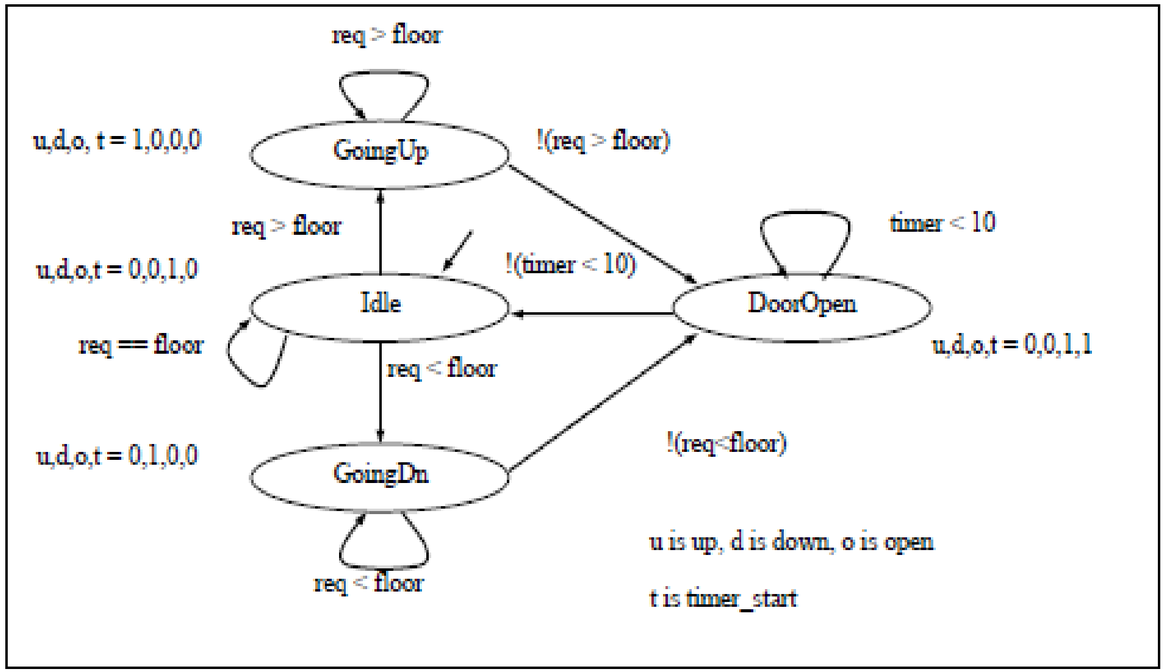
Events:

Several events control the transitions between these states:

- req > floor
- req < floor
- req == floor
- !(req > floor)
- !(req < floor)
- timer < 10
- !(timer < 10)

FSM Model – Example 3

- Initially, the elevator is in the **Idle** state. If a request is made for a floor higher than the current floor (req > floor), the elevator transitions to the **GoingUp** state and begins moving upward.
- If a request is made for a floor lower than the current floor (req < floor), the elevator transitions to the **GoingDown** State and moves downward.



State Diagram

- If the current floor is the requested floor (req == floor), it immediately transitions to the **DoorOpen** state to allow entry or exit.

FSM Model – Example 3

- While in the **GoingUp** state, the elevator keeps moving upward as long as there is a higher request (req > floor). Once there are no more higher requests (!(req > floor)), it transitions to the **DoorOpen** state.
- Similarly, in the **GoingDown** state, the elevator continues moving down while there is a lower request (req < floor). Once there are no more lower requests (!(req < floor)), it moves to the **DoorOpen** state.
- In the **DoorOpen** state, the door remains open for a minimum of 10 seconds. As long as the timer is less than 10 seconds (timer < 10), the elevator stays in this state. Once the timer reaches 10 seconds or more (!(timer < 10)), the elevator returns to the **Idle** state, waiting for the next request.